

## PRACTICAL WEEK-9

Implement a file copy utility using low-level Linux file 1/0 system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Code:

```
GNU nano 8.7.1 lowlevel_copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int src, dest;
    char buffer[1024];
    ssize_t bytes;

    src = open("input.txt", O_RDONLY);

    if (src == -1) {
        perror("Error opening source file");
        return 1;
    }

    dest = open("copy.txt",
               O_WRONLY | O_CREAT | O_TRUNC,
               0644);

    if (dest == -1) {
        perror("Error opening destination file");
        close(src);
        return 1;
    }

    while ((bytes = read(src, buffer, sizeof(buffer))) > 0) {
        write(dest, buffer, bytes);
    }

    close(src);
    close(dest);

    printf("File copied successfully.\n");

    return 0;
}

[ Read 37 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Output:

```
root@Mani: ~$/practical$ gcc lowlevel_copy.c -o lowlevel_copy
root@Mani: ~$/practical$ ./lowlevel_copy
File copied successfully.
root@Mani: ~$/practical$ cat copy.txt
Hello Ubuntu!
This is a file copy experiment.
We are using Linux system calls.
root@Mani: ~$/practical$ |
```

```
GNU nano 8.7.1                                seek.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char ch;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    // Move file pointer to the beginning
    lseek(fd, 0, SEEK_SET);

    read(fd, &ch, 1);

    printf("First character: %c\n", ch);

    close(fd);

    return 0;
}
```

```
root@Mani: ~$/practical$ nano seek.c
root@Mani: ~$/practical$ gcc seek.c -o seek
root@Mani: ~$/practical$ ./seek
First character: H
root@Mani: ~$/practical$ nano std_copy.c
root@Mani: ~$/practical$ |
```

```
GNU nano 8.7.1                                std_copy.c *
#include <stdio.h>

int main() {
    FILE *src, *dest;
    char buffer[1024];
    size_t bytes;

    src = fopen("input.txt", "rb");

    if (src == NULL) {
        perror("Error opening source file");
        return 1;
    }

    dest = fopen("std_copy.txt", "wb");

    if (dest == NULL) {
        perror("Error opening destination file");
        fclose(src);
        return 1;
    }

    while ((bytes = fread(buffer, 1, sizeof(buffer), src)) > 0) {
        fwrite(buffer, 1, bytes, dest);
    }

    fclose(src);
    fclose(dest);

    printf("File copied using standard library.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
root@Mani: ~$/practical$ nano std_copy.c
root@Mani: ~$/practical$ gcc std_copy.c -o std_copy
root@Mani: ~$/practical$ ./std_copy
File copied using standard library.
root@Mani: ~$/practical$ cat std_copy.txt
Hello Ubuntu!
This is a file copy experiment.
We are using Linux system calls.
root@Mani: ~$/practical$ |
```

```
root@Mani: ~$/practical$ time ./lowlevel_copy
File copied successfully.

real    0m0.010s
user    0m0.001s
sys     0m0.001s
root@Mani: ~$/practical$ time ./std_copy
File copied using standard library.

real    0m0.003s
user    0m0.001s
sys     0m0.003s
root@Mani: ~$/practical$ |
```

Report: A file-copy utility was successfully implemented using low-level Linux I/O system calls. The program used `open()`, `read()`, `write()`, `lseek()`, and `close()` to access files. A second version using `fopen()`, `fread()`, and `fwrite()` was also implemented. Execution time was compared using the `time` command, showing that performance depends on file size and buffering.

Task-2: Develop a program that redirects standard input and Output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

```
GNU nano 8.7.1          redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int input, output;

    input = open("input.txt", O_RDONLY);

    if (input == -1) {
        perror("input.txt");
        return 1;
    }

    output = open("output.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (output == -1) {
        perror("output.txt");
        close(input);
        return 1;
    }

    // Redirect standard input
    dup2(input, STDIN_FILENO);

    // Redirect standard output
    dup2(output, STDOUT_FILENO);

    close(input);
    close(output);

    // This now reads from input.txt
    // and writes to output.txt
    execlp("sort", "sort", NULL);
}
```

<sup>^</sup>G Help      <sup>^</sup>O Write Out    <sup>^</sup>F Where Is    <sup>^</sup>K Cut        <sup>^</sup>T E  
<sup>^</sup>X Exit      <sup>^</sup>R Read File   <sup>^</sup>\ Replace    <sup>^</sup>U Paste      <sup>^</sup>J J

```
root@Mani: ~/practical$ nano input.txt
root@Mani: ~/practical$ nano redirect.c
root@Mani: ~/practical$ gcc redirect.c -o redirect
root@Mani: ~/practical$ ./redirect
root@Mani: ~/practical$ cat output.txt
apple
banana
grape
orange
root@Mani: ~/practical$ |
```

Report: The program successfully demonstrated standard input and output redirection using `dup2()`. The input of the `sort` command was redirected from a file instead of the keyboard. Its output was redirected from the terminal to another file. This demonstrated the basic mechanism used by Linux shells to implement `<` and `>` redirection.